

1. Visión General

El proyecto es un sistema de **Video On Demand (VOD)** basado en el protocolo **MPEG-DASH**. Su arquitectura está desacoplada en dos aplicaciones web desplegadas en el mismo servidor(Glassfish):

1. **API-RESTStreaming (Backend):** Se encarga de leer el disco duro y servir los archivos de vídeo (manifiesto .mpd y segmentos .m4s) mediante HTTP. También realiza conexiones con la base de datos, en este caso solo guardamos los usuarios y si tienen categoría de administrador.
 2. **ClienteStreaming (Frontend):** Es la interfaz web que consume la API y reproduce el vídeo utilizando la librería `dash.js`.
-

2. Módulo Backend: API - RESTStreaming

Este proyecto actúa como el servidor de datos. No tiene interfaz gráfica, solo responde peticiones HTTP.

`JakartaEE91Resource.java`

- **Responsabilidad:** Controlador REST (Endpoint).
- **Función Principal:** Recibir las peticiones del navegador y entregar los archivos físicos.
- **Métodos Clave:**
 - `@GET @path("list"):`
devuelve un json con la lista de los nombres de los videos procesados en el servidor, usa la clase `GestionCarpeta` para ello.
 - `@POST @path("upload"):`
recibe un video en formato `octet_stream` lo procesa con el script `ffmpeg` que esta en la clase `conversor.java` de `utils`, cuando acaba sino ha habido ningun problema devuelve la peticion con el codigo 200 "ok".
 - `@DELETE @path("delete/{nombre}"):`
elimina un video de la carpeta de videos, utiliza la clase `GestionCarpeta`.
 - `@PUT @path("rename"):`
recibe el nombre viejo y nuevo de un archivo y lo modifica.
 - `@POST @path("login"):`
recibe un intento de login y devuelve en un json si es correcto o no y en caso de ser correcto si tiene permisos de administrador.

- **@PUT @path("register"):**

recibe una petición para registrar un nuevo usuario en la base de datos y comprueba si existe o no, en caso de no existir lo crea que el usuario y contraseña facilitados.

- **@GET @path(stream/{videoName}/{fileName}):**

devuelve archivos mpd o m4s para realizar el streaming.

Utils/Conversor.java

- **Responsabilidad:** complementar la operación de subida de videos via http.
- **Función Principal:** codificar videos de formato mp4 a mpd/m4s
- **Métodos Clave:**

`ejectutarScriptConversion/generateDash`

Aqui esta la configuracion de conversion de ffmpeg a la vez que estan definidas las carpetas de entrada salida donde estan o estaran los archivos.

Utils/DBConnection.java

- **Responsabilidad:** Conexion a la base de datos
- **Función Principal:** verificar logins, crear usuarios y verificar que existen tambien se devuelve si el usuario tiene permisos de adminisitrador.
- **Métodos Clave:**

boolean checkLogin

verifica si el login es correcto, o no.

boolean existeUsuario

verifica si un usuario existe o no.

void registrarUsuario

Registra un nuevo usuario en la base de datos

boolean esAdmin

comprueba si un usuario tiene permisos de administración

Utils/GestionCarpeta.java

- **Responsabilidad:** facilitar el codigo de otras clases.
- **Función Principal:** Realizar operaciones de gestion de archivos
- **Métodos Clave:**

void vaciarCarpetaInput()

vacía la carpeta raíz de los archivos mp4 para evitar que se llene de archivos temporales.

String guardarVideoEnInput(Part filePart, String nombreArchivo)

guarda temporalmente el video proveniente del formulario en la carpeta raiz de los mp4. Devuelve la ruta donde se ha guardado el video.

List<String> getListaVideos()

devuelve la lista de videos procesados, es decir la lista de carpetas de la carpeta output del conversor.

Boolean borrarVideo(String nombreCarpeta)

borra la carpeta y todo su contenido de forma recursiva, se usa para facilitar el borrado de videos.

boolean renombrarVideo(String carpetaActual, String nuevoNombreUsuario)

renombra un carpeta de un video con un nuevo nombre proporcionado por un usuario administrador.

borrarRecursivamente(File archivo)

funcion auxiliar de borrarVideo

procesarNuevoVideo(jakarta.servlet.http.Part filePart, String nombreUsuario)

procesa un nuevo video con ayuda del conversor.

Utils/VideoTranscoder.java

- **Responsabilidad:** ninguna
- **Función Principal:** facilitar la conversion de videos sin necesidad de depender del servidor, se puede hacer de forma manual
- **Métodos Clave:**

void main()

pasa los de videos de una carpeta input de mp4 a otra carpeta output a formato m4s/mpd.

3. Módulo Frontend: ClienteStreaming

Este proyecto es lo que ve el usuario final. Gestiona la navegación y el reproductor.

Filters/admin o Filters/Sesion

- **Responsabilidad:** limitar el acceso a usuarios
- **Función:**
Controlar el acceso a ciertos jsp/servlets

Utils/ClienteAPI

- **Responsabilidad:** Comunicacion via http con REST
- **Función:**
facilitar el codigo de los servlets y orquestrar la comunicación con la API rest.
- **Métodos Clave:**

String get(String endpoint)

Realiza petición get y devuelve el cuerpo como string.

boolean delete(String endpoint)

Realiza petición delete de un video, devuelve true si se ha podido borrar

boolean postStream(String endpoint, InputStream fileStream)

Realiza una petición post y devuelve true si se ha podido realizar.

List<String> jsonArrayToList(String json)

Convierte un json a una lista de strings que soporta comas dentro de los nombres.

int checkLogin(String username, String password)

Verifica credenciales de un login, puede devolver 0,1,-1 (normal,administrador,error)

boolean register(String username, String password)

Registra un nuevo usuario. Devuelve true si ha podido realizar.

boolean renombrarVideo(String nombreActual, String nuevoNombre)

Renombra un video, devuelve true si se pudo renombrar.

boolean uploadVideo(File archivoVideo, String nombreVideo)

sube un video, devuelve true cuando se ha acabado de procesar. El timeout lo hacemos adaptativo al tamaño del archivo con una estimacion a la alta de cuanto debería tardar en funcion de los mb de video.

js/JDashPlayer.js

- **Responsabilidad:** Lógica del Cliente (Wrapper de Dash.js).
- **Función:** Es una clase JavaScript personalizada que envuelve la librería oficial `dash.all.min.js`.
 - `init()`: Inicializa el reproductor y lo conecta al elemento HTML.
 - `setQuality(index)`: Fuerza al reproductor a cambiar a una calidad específica (360p, 720p, etc.).
 - `setAuto()`: Activa el algoritmo de bitrate adaptativo (ABR) para que la calidad cambie según la velocidad de internet.
 - Monitoreo: Actualiza en tiempo real los datos del dashboard (Bitrate actual, resolución).

css/estilo.css

- **Responsabilidad:** Estilos visuales.
- **Función:** Define la apariencia del reproductor oscuro, los botones de calidad, la disposición del dashboard y la responsividad de la página.

Servlets/AdminVideos

Responsabilidad: Controlador principal para la gestión y visualización del panel de administración de videos.

Función: Su objetivo es preparar los datos necesarios para la vista de administración (`admin.jsp`). Orquesta la obtención del listado de videos desde la API externa utilizando `ClienteAPI`, convierte los datos recibidos y gestiona la visualización de mensajes de retroalimentación (alertas) para el usuario.

Métodos Clave:

- `void processRequest(HttpServletRequest request, HttpServletResponse response)` Ejecuta la lógica central del servlet:
 1. Llama a `ClienteAPI.get("list")` para obtener el JSON de videos.
 2. Convierte el JSON a una lista Java (`List<String>`).
 3. Establece los atributos `listaVideosAdmin` (datos) y `mensajeAlerta` (notificaciones UI) en el objeto request.
 4. Despacha la petición hacia `admin.jsp`.

- `doGet(HttpServletRequest request, HttpServletResponse response)` Intercepta las peticiones HTTP GET y delega el procesamiento a `processRequest`.
- `doPost(HttpServletRequest request, HttpServletResponse response)` Intercepta las peticiones HTTP POST y delega el procesamiento a `processRequest`.

Servlets/Eliminar

Responsabilidad: Gestión de la eliminación de recursos (videos) mediante comunicación con la API REST.

Función: Recibe la solicitud de borrado de una carpeta/video específico, delega la ejecución real a la API externa utilizando `ClienteAPI` y gestiona la navegación de retorno al panel de administración con indicadores de estado (éxito o error).

Métodos Clave:

- `void processRequest(HttpServletRequest request, HttpServletResponse response)` Contiene la lógica de negocio:
 1. Captura el parámetro `carpeta` de la solicitud.
 2. Invoca `ClienteAPI.delete("delete/" + nombreCarpeta)` para realizar el borrado en el servidor remoto.
 3. Evalúa el resultado booleano y redirige a `AdminVideos` concatenando el parámetro `msg` (borrado o error) para informar al usuario.
- `doPost(HttpServletRequest request, HttpServletResponse response)` Punto de entrada principal para las solicitudes de eliminación. Delega en `processRequest`.
- `doGet(HttpServletRequest request, HttpServletResponse response)` Implementa una medida de seguridad. Redirige inmediatamente a `AdminVideos` sin procesar ninguna lógica de borrado, evitando que se puedan eliminar elementos accidentalmente accediendo a la URL directa desde el navegador.

Servlets/ListaVideos

Responsabilidad: Controlador encargado de la gestión de contenidos para la vista pública (menú de usuario).

Función: Su función principal es recuperar el catálogo de videos disponibles consumiendo el servicio REST externo. Actúa como intermediario para que la vista (`menu.jsp`) reciba una lista de datos agnóstica de su origen (API remota), permitiendo la visualización del contenido multimedia.

Métodos Clave:

- `void processRequest(HttpServletRequest request, HttpServletResponse response)` Ejecuta el flujo de carga de datos:

1. Realiza una petición HTTP GET al endpoint `list` mediante `ClienteAPI.get("list")`.
 2. Transforma la respuesta JSON cruda en una estructura de datos Java (`List<String>`) utilizando el parser de `ClienteAPI`.
 3. Inyecta la lista resultante en la petición bajo el atributo `"listaVideos"`.
 4. Redirige el flujo al JSP de presentación (`menu.jsp`).
- `doGet(HttpServletRequest request, HttpServletResponse response)` Maneja las peticiones GET entrantes y delega la operación a `processRequest`.
 - `doPost(HttpServletRequest request, HttpServletResponse response)` Maneja las peticiones POST entrantes y delega la operación a `processRequest`.

Servlets/Login

Responsabilidad: Gestión del proceso de autenticación, control de acceso y creación de sesiones de usuario.

Función: Actúa como la puerta de entrada a la aplicación. Recibe las credenciales del formulario, delega la validación de seguridad a la clase auxiliar `ClienteAPI` (que consulta el servicio externo) y gestiona el ciclo de vida de la sesión HTTP, asignando roles ("admin" o "user") y redirigiendo al usuario según el resultado de la validación.

Métodos Clave:

- `void processRequest(HttpServletRequest request, HttpServletResponse response)` Orquesta el flujo de login:
 1. Captura los parámetros `usuario` y `pass` del formulario.
 2. Invoca `ClienteAPI.checkLogin(usuario, pass)` para validar las credenciales remotamente.
 3. **Si la validación es exitosa (`resultado != -1`):**
 - Inicia una nueva `HttpSession`.
 - Evalúa el entero retornado: si es 1 asigna el rol "**admin**", en caso contrario asigna "**user**".
 - Redirige al usuario al servlet `ListaVideos`.
 4. **Si la validación falla:**
 - Inyecta un mensaje de error en el atributo `"error"`.
 - Devuelve la vista `login.jsp` para que el usuario reintente.
- `doGet(HttpServletRequest request, HttpServletResponse response)` Fuerza la redirección al formulario de login (`login.jsp`). Evita que se

procesen intentos de autenticación mediante parámetros en la URL por motivos de seguridad.

- `doPost(HttpServletRequest request, HttpServletResponse response)` Recibe los datos del formulario de login y delega el procesamiento a `processRequest`.

Servlets/Logout

Responsabilidad: Finalización segura de la sesión del usuario y redirección de flujo.

Función: Su única función es garantizar que la sesión activa del usuario sea destruida en el servidor, eliminando cualquier atributo de seguridad (como el usuario o el rol) y devolviendo al usuario a la pantalla de inicio de sesión.

Métodos Clave:

- `void processRequest(HttpServletRequest request, HttpServletResponse response)` Ejecuta la lógica de salida:
 1. Intenta obtener la sesión actual sin crear una nueva (`request.getSession(false)`).
 2. Si la sesión existe, invoca `session.invalidate()` para destruirla y liberar los recursos asociados.
 3. Redirige al navegador a la página `login.jsp`.
- `doGet(HttpServletRequest request, HttpServletResponse response)` Permite cerrar sesión mediante una simple petición GET (clic en un enlace) y delega a `processRequest`.
- `doPost(HttpServletRequest request, HttpServletResponse response)` Permite cerrar sesión mediante envío de formulario (si fuera necesario) y delega a `processRequest`.

Servlets/Modificar

Responsabilidad: Gestión del flujo de edición (renombrado) de los recursos de video existentes.

Función: Opera en dos fases: primero, intercepta la solicitud de edición para cargar el formulario con los datos actuales del video; segundo, procesa los datos enviados por dicho formulario, delegando la ejecución del cambio a la API REST y gestionando la redirección según el resultado de la operación.

Métodos Clave:

- `doGet(HttpServletRequest request, HttpServletResponse response)` **Fase de Visualización:**
 1. Captura el parámetro `v` de la URL (el nombre actual de la carpeta).
 2. Valida que el parámetro no sea nulo; si lo es, redirige al listado principal.

3. Prepara el atributo `carpetaParaEditar` y despacha la solicitud a `modificar.jsp` para que el usuario vea el formulario con el nombre actual pre-cargado.
- `doPost(HttpServletRequest request, HttpServletResponse response)` **Fase de Procesamiento:**
 1. Recoge los parámetros del formulario (`nombreOriginal` y `nuevoNombre`).
 2. Realiza una validación básica para asegurar que los campos no estén vacíos.
 3. Invoca a `ClienteAPI.renombrarVideo(nombreOriginal, nuevoNombre)` para ejecutar la actualización (PUT) en el servidor remoto.
 4. Redirige a `AdminVideos` adjuntando un parámetro `msg` (`modificado`, `errorModificar` o `camposVacios`) para notificar al usuario el resultado de la operación.

Servlets/Registrar

Responsabilidad: Gestión del alta de nuevos usuarios en el sistema.

Función: Actúa como intermediario entre el formulario de registro y la API. Su función principal es validar la integridad de los datos de entrada (como la coincidencia de contraseñas) antes de consumir recursos de la API. Si las validaciones locales pasan, solicita la creación del usuario al servicio externo y gestiona la respuesta, permitiendo el acceso o informando de errores (como usuario duplicado).

Métodos Clave:

- `void processRequest(HttpServletRequest request, HttpServletResponse response)` Ejecuta el flujo de registro:
 1. **Recogida de datos:** Captura usuario y contraseñas.
 2. **Validación Local:** Verifica que los campos no estén vacíos y que las contraseñas coincidan *antes* de llamar a la API (ahorrando tráfico innecesario).
 3. **Llamada a API:** Si los datos son válidos localmente, invoca `ClienteAPI.register(usuario, pass1)`.
 4. **Gestión de Respuesta:**
 - **Éxito:** Redirige a `login.jsp` con un indicador de éxito.
 - **Error:** Si la API devuelve `false` (por ejemplo, usuario existente o error 409), define un mensaje de error, restaura el nombre de usuario ingresado (`usuarioPrevio`) para mejorar la UX y devuelve la vista `registrar.jsp`.
- `doGet(HttpServletRequest request, HttpServletResponse response)` Protege el servlet contra ejecuciones accidentales vía URL. Redirige al usuario al formulario `registrar.jsp` para que ingrese los datos correctamente.

- `doPost(HttpServletRequest request, HttpServletResponse response)` Punto de entrada del formulario. Delega el procesamiento a `processRequest`.

Servlets/SubirVideo

Responsabilidad: Gestión de la carga (upload) de nuevos archivos de video al sistema mediante formularios `multipart/form-data`.

Función: Actúa como un "puente" de transferencia. Su función es recibir el flujo de datos binarios (stream) del navegador, convertirlo temporalmente en un archivo físico manejable por el servidor local, y delegar la subida final a la API REST externa. Además, impone límites estrictos al tamaño de los archivos para proteger la memoria del servidor y garantiza la limpieza del disco tras la operación.

Métodos Clave:

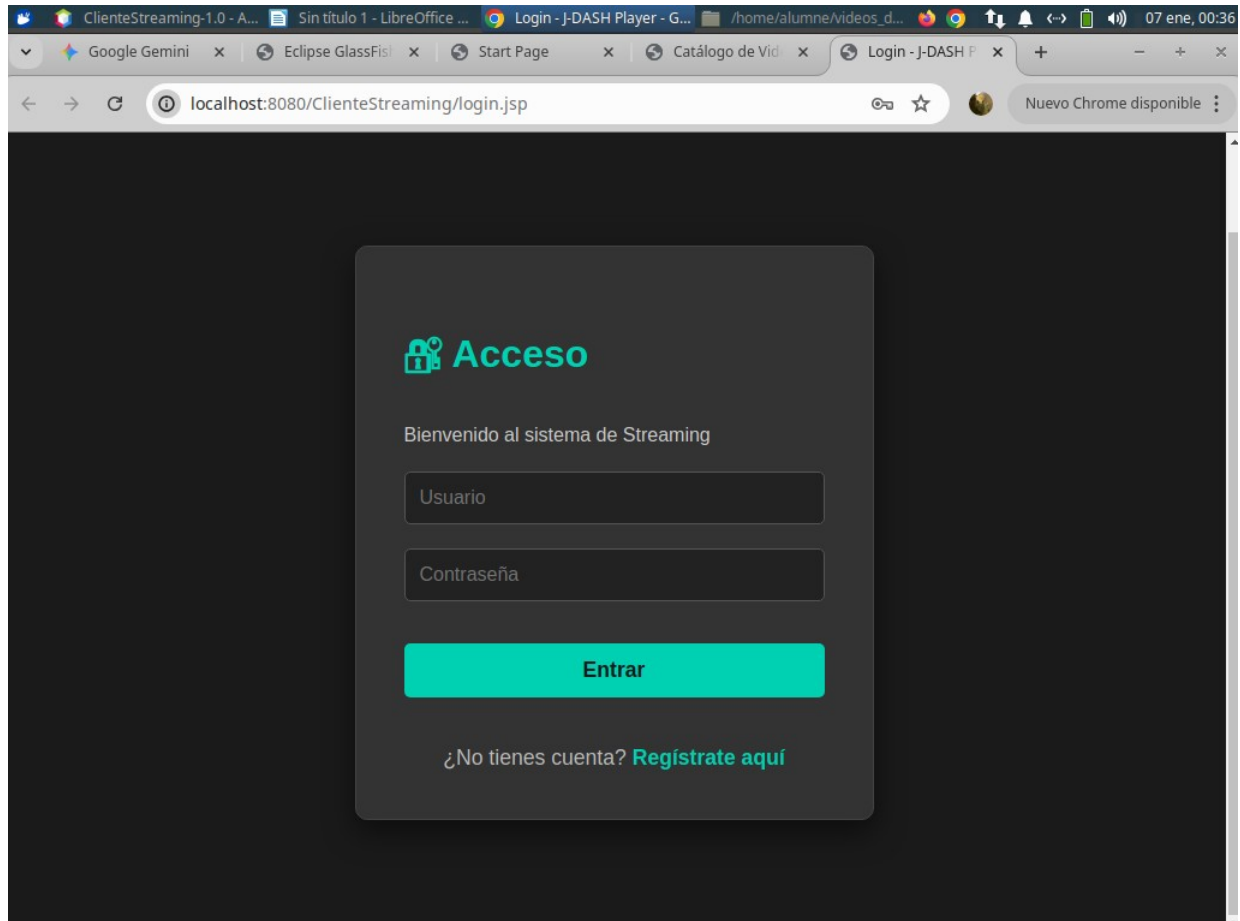
- `doGet(HttpServletRequest request, HttpServletResponse response)` Simplemente despacha la solicitud a la vista `subirVideo.jsp` para mostrar el formulario de carga al usuario.
- `doPost(HttpServletRequest request, HttpServletResponse response)` Contiene la lógica compleja de subida:
 1. **Recepción y Validación:** Utiliza `request.getPart` para obtener el archivo de video y valida que los datos no sean nulos.
 2. **Creación de Archivo Puente (Temp):** Como el objeto `Part` (stream) no se puede enviar directamente a través de todas las librerías HTTP, crea un archivo temporal en el disco local (`File.createTempFile`) y copia el contenido del video allí.
 3. **Transferencia a API:** Invoca a `ClienteAPI.uploadVideo(tempFile, nombreUsuario)`, pasando el archivo físico temporal.
 4. **Gestión de Respuesta:** Redirige al panel de administración si la subida es exitosa, o devuelve un error al formulario si falla.
 5. **Limpieza (Finally):** En el bloque `finally`, asegura que el archivo temporal creado en el paso 2 sea eliminado (`tempFile.delete()`) para evitar llenar el disco del servidor con "basura" de subidas anteriores.

Notas Adicionales:

- **Configuración Multipart:** Utiliza la anotación `@MultipartConfig` para definir límites físicos:
 - Umbral de memoria: 2MB.
 - Tamaño máximo de archivo: 500MB.
 - Tamaño máximo de petición: 600MB.

Resumen de Vistas (JSPs)

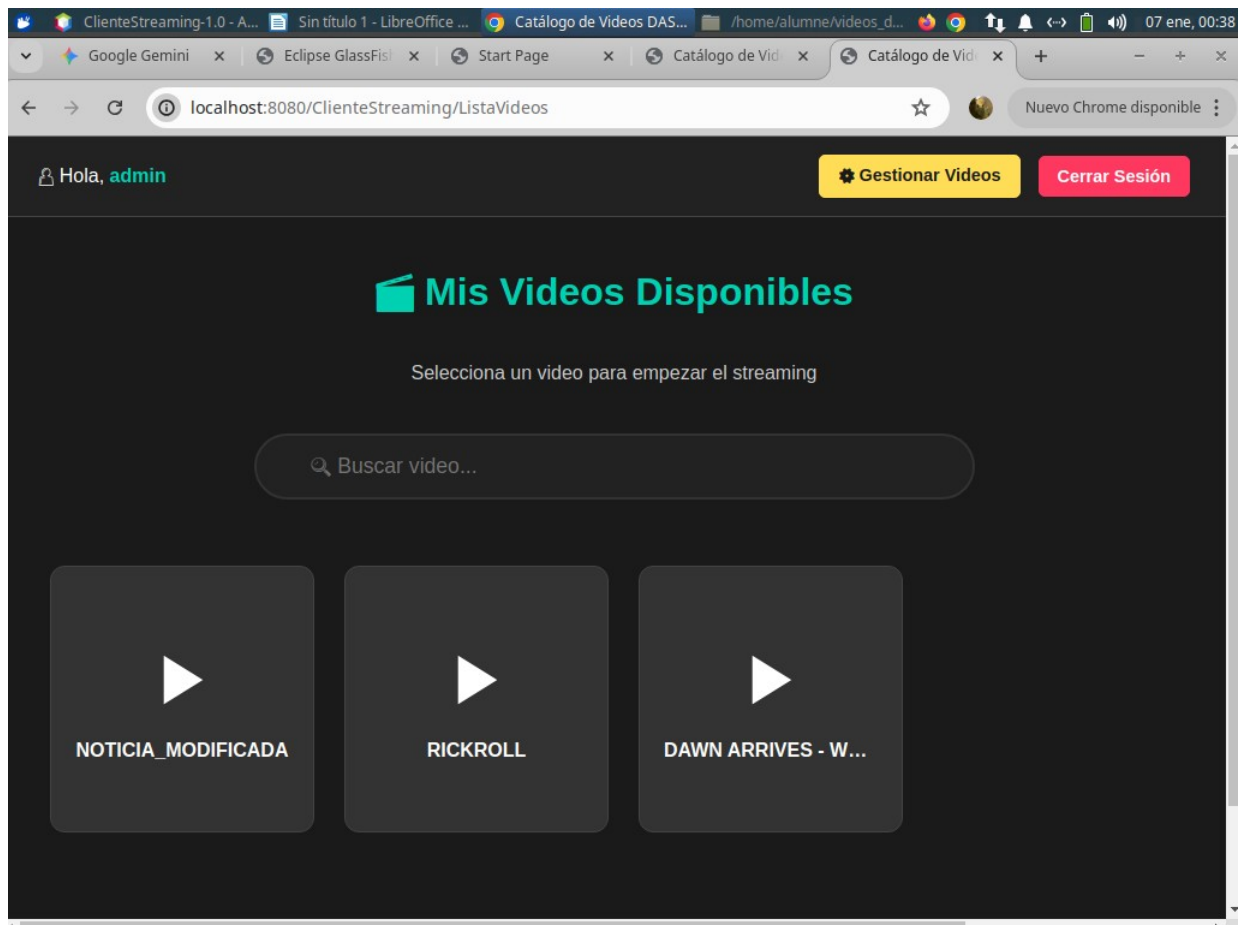
- **login.jsp:** Pantalla de inicio de sesión. Gestiona la autenticación de usuarios y muestra alertas de error/éxito.



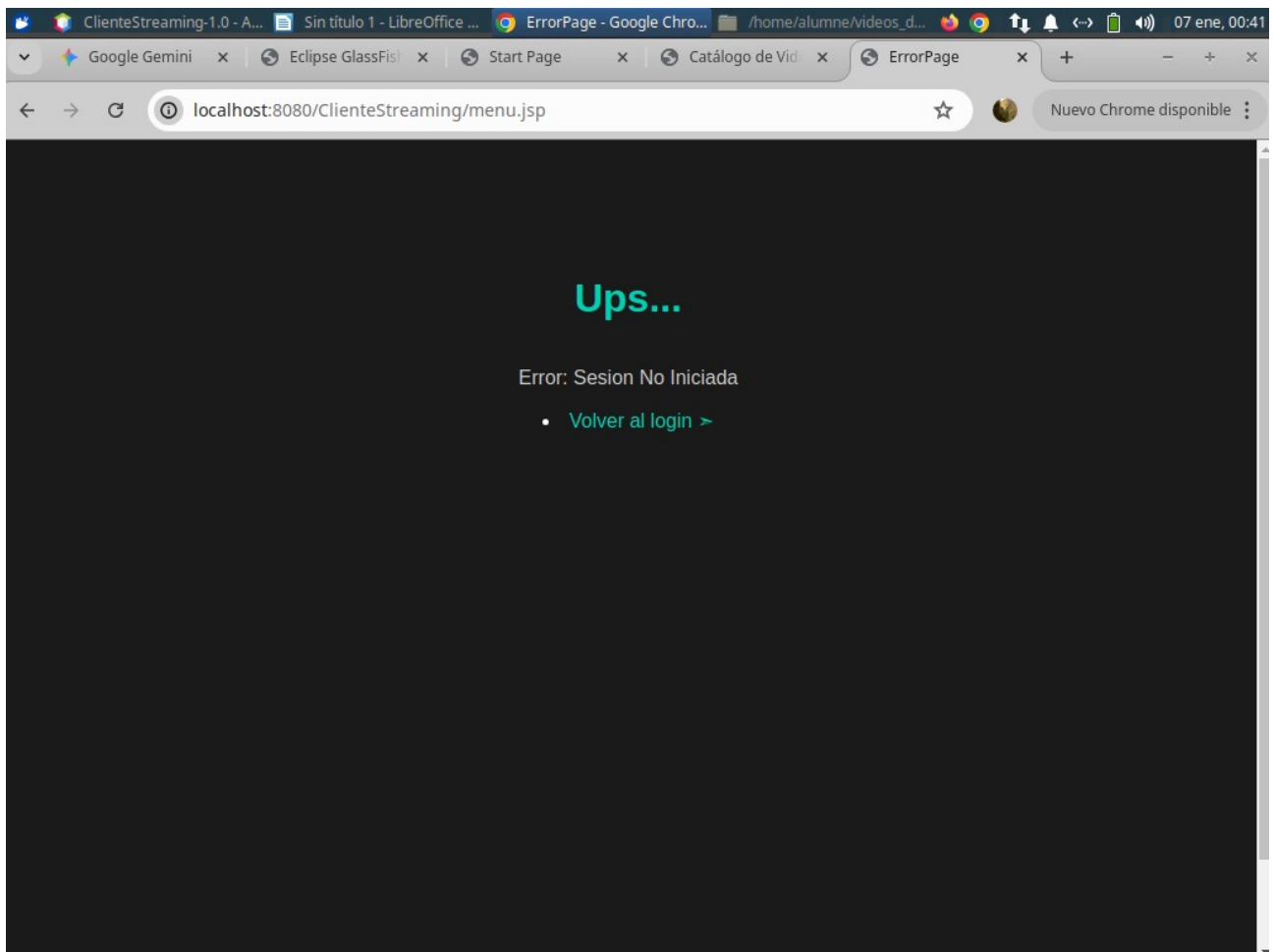
- **registrar.jsp**: Formulario de registro. Permite crear nuevos usuarios con validación de contraseñas y usabilidad mejorada (persistencia de datos en error).

The screenshot shows a web browser window with the address bar displaying `localhost:8080/ClienteStreaming/registrar.jsp`. The page has a dark theme. At the top, there's a header with the title "Registro" in green, accompanied by a document icon. Below the header, the text "Crea tu cuenta nueva" is displayed. The form consists of three input fields: "Nombre de Usuario" with a placeholder "Elige tu usuario", "Contraseña" with a placeholder "Mínimo 4 caracteres", and "Repetir Contraseña" with a placeholder "Confirma tu contraseña". A prominent green button labeled "Registrarse" is positioned below the fields. At the bottom of the form, there is a link that says "¿Ya tienes cuenta? Inicia sesión". The browser's tab bar shows several open tabs, including "Google Gemini", "Eclipse GlassFish", "Start Page", "Catálogo de Vid...", and "Registro - J-DASH". The system tray at the top right indicates the date and time as "07 ene, 00:37".

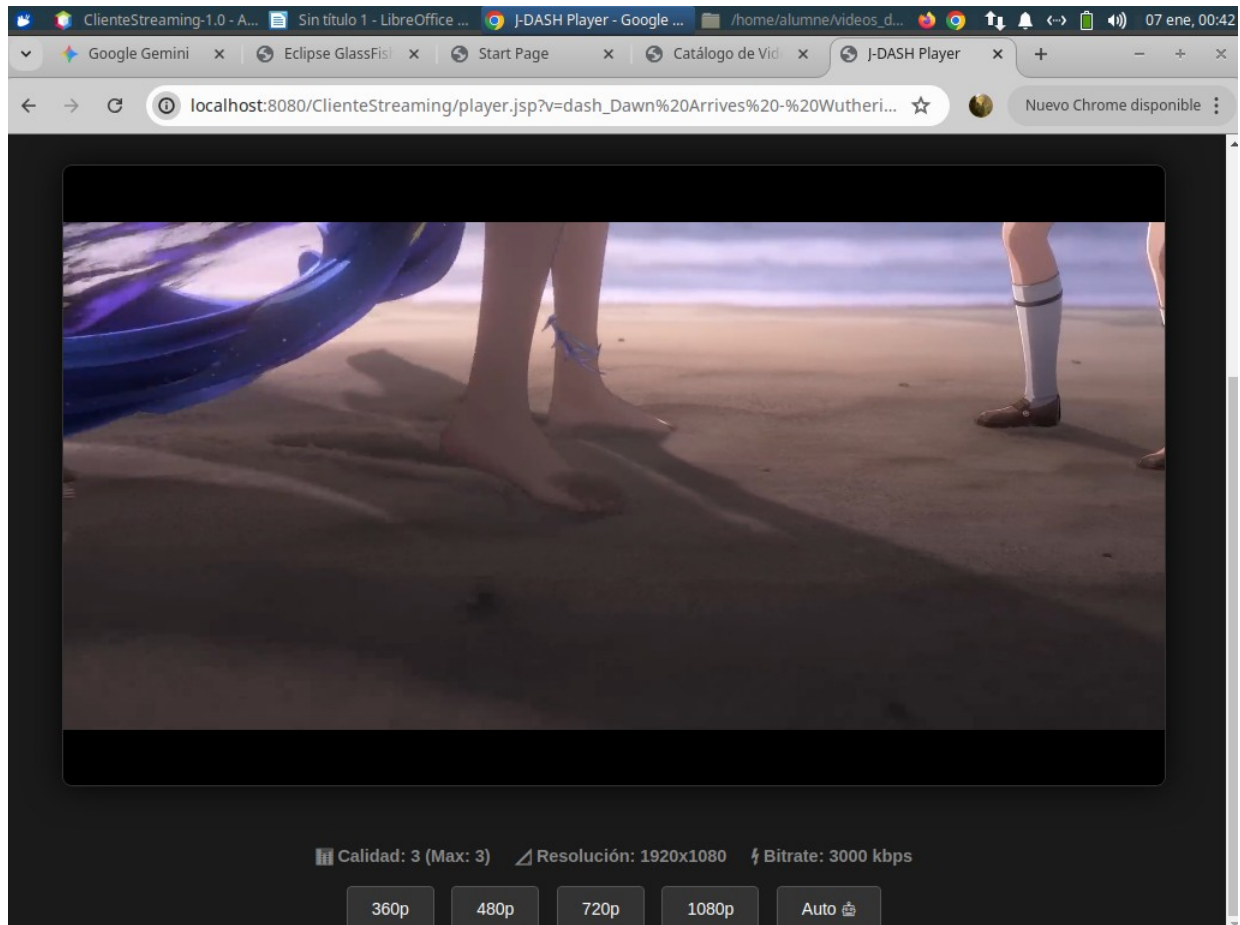
- **menu.jsp**: Dashboard principal. Muestra el catálogo de videos en una cuadrícula (grid) con buscador en tiempo real y opciones según el rol (Admin/Usuario). El apartado Gestionar Videos solo aparece si el usuario tiene el rol de admin



- **error.jsp:** Página de error genérica. Muestra mensajes dinámicos y ofrece un botón de retorno inteligente (al login o al menú) según el contexto.



- **player.jsp:** El reproductor de video. Integra `dash.js` y mi `JDashPlayer.js` para reproducir el contenido, permitiendo cambio de calidad manual/automático y ver estadísticas.



- **admin.jsp**: Panel de gestión. Tabla CRUD donde el administrador ve todos los videos, con accesos directos a modificar, eliminar y subir contenido.

ClienteStreaming-1.0 - A... Sin título 1 - LibreOffice ... Panel de Administración ... /home/alumne/videos_d... 07 ene, 00:43

Google Gemini x Eclipse GlassFish x Start Page x Catálogo de Vid x Panel de Admini x

localhost:8080/ClienteStreaming/AdminVideos

MODO ADMIN: admin Cerrar Sesión

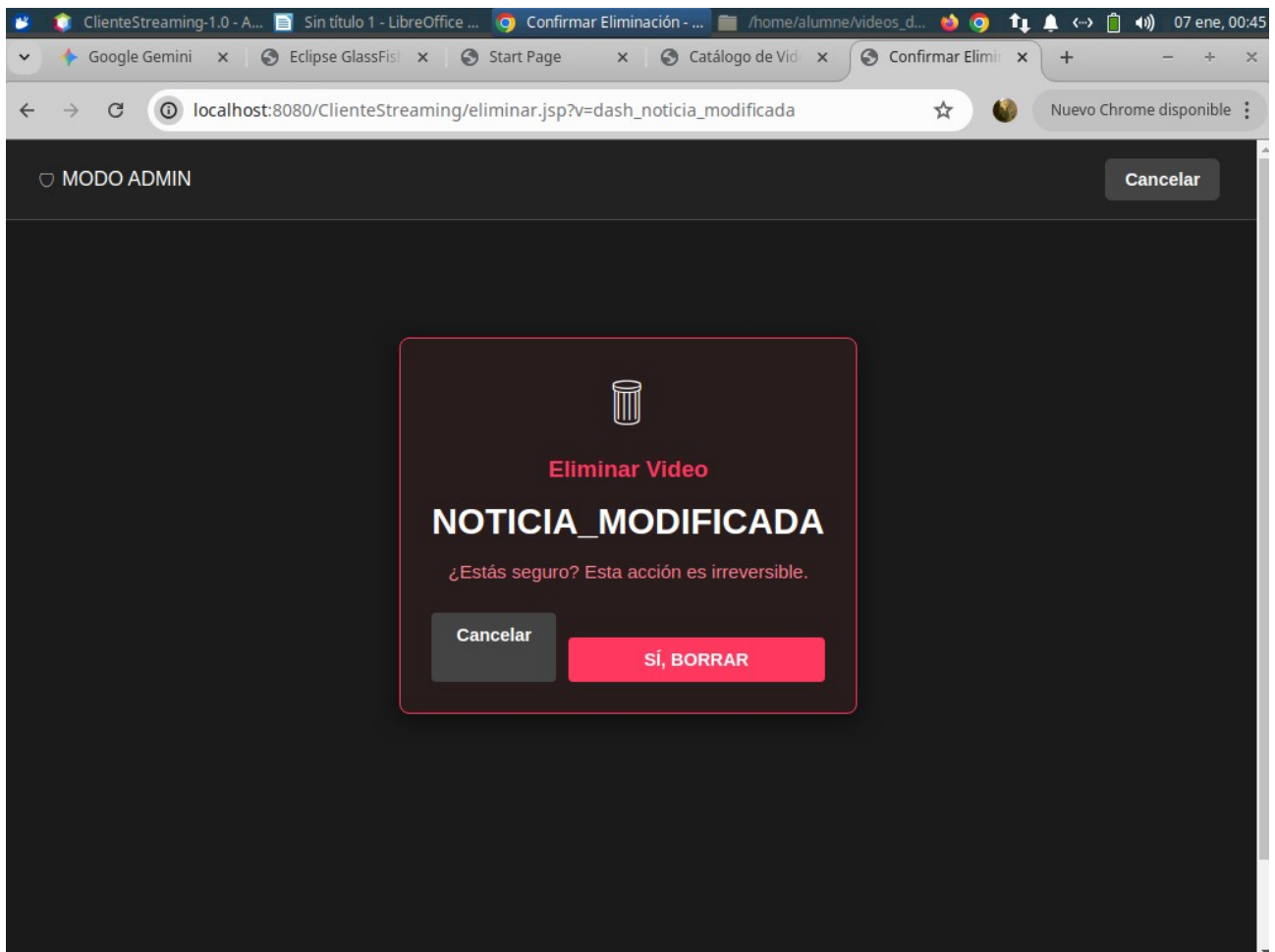
⚙️ GESTIÓN DE CONTENIDO

📁 Subir Nuevo Video

NOMBRE DEL VIDEO	ID SISTEMA (CARPETA)	ACCIONES
📁 NOTICIA_MODIFICADA	dash_noticia_modificada	➡ Modificar 🗑 Eliminar
📁 RICKROLL	dash_rickroll	➡ Modificar 🗑 Eliminar
📁 DAWN ARRIVES - WUTHERING WAVES (1080P, H264)	dash_Dawn Arrives - Wuthering Waves (1080p, h264)	➡ Modificar 🗑 Eliminar

← Volver al Menú Principal

- **eliminar.jsp**: Confirmación de borrado. "Zona de peligro" visual para evitar borrados accidentales de videos.



- **modificar.jsp**: Edición de metadatos. Formulario para renombrar videos, mostrando el ID interno (solo lectura) y validando la entrada.

The screenshot shows a web browser window with the address bar displaying `localhost:8080/ClienteStreaming/Modificar?v=dash_noticia_modificada`. The browser tabs include 'ClienteStreaming-1.0 - A...', 'Sin título 1 - LibreOffice ...', 'Modificar Video - Google...', and '/home/alumne/videos_d...'. The page title is 'MODIFICAR VIDEO'. The interface has a dark theme. At the top left, it says 'MODO ADMIN' with a shield icon. At the top right, there is a 'Cancelar' button. The main content area features a large teal header with a pencil icon and the text 'EDITAR VIDEO'. Below this, there is a form with two input fields. The first field is labeled 'Nombre Actual (ID Sistema):' and contains the text 'dash_noticia_modificada'. The second field is labeled 'Nuevo Nombre:' and contains the text 'Ej: Matrix Reloaded'. Below the second field, there is a note: 'El sistema añadirá 'dash_' y guiones bajos automáticamente.' At the bottom of the form, there are two buttons: 'Cancelar' and 'Guardar Cambios' (which is highlighted in teal).

MODO ADMIN

Cancelar

 EDITAR VIDEO

Nombre Actual (ID Sistema):

dash_noticia_modificada

Nuevo Nombre:

Ej: Matrix Reloaded

El sistema añadirá 'dash_' y guiones bajos automáticamente.

Cancelar

 Guardar Cambios

- **subirVideo.jsp**: Carga de contenido. Formulario **multipart** para enviar archivos MP4. Incluye un **overlay de carga (spinner)** con JavaScript para bloquear la pantalla mientras el servidor realiza la conversión pesada a DASH.

